

Fundamentos de Redes de Computadores



Definição do Trabalho Final

A definição do trabalho consiste em implementar uma aplicação que simula o funcionamento de uma rede local em anel. A aplicação deverá implementar a transmissão de mensagens entre as máquinas que compõem o anel, utilizando o protocolo UDP como transporte.

A topologia em anel é construída a partir de uma lista ordenada de máquinas ativas em ordem alfabética de apelido. Cada máquina identifica seu sucessor como o próximo elemento da lista, com comportamento circular (o último conecta-se ao primeiro). Um *token* circula pelo anel controlando quem pode transmitir: somente a máquina que detém o token pode enviar dados.

O programa possui três tipos de pacotes: **controle** (DISCOVER e HELLO), **token** e **dados**.

Toda a comunicação utiliza UDP na **porta 6000**: broadcast para DISCOVER e HELLO; unicast para token e dados.

Arquivo de configuração

Ao iniciar, o programa lê um arquivo de configuração cujo formato fica a critério do grupo, desde que contenha:

- **Apelido da máquina** (letras A, B, C...);
 - **Delay do token e dos dados** — tempo de espera antes de enviar uma mensagem ou passar o token, para controlar artificialmente o ritmo da rede;
 - **Probabilidade de inserção de erro** — probabilidade com que o módulo de falhas corrompe mensagens antes do envio;
 - **Timeout do token** — tempo máximo sem o token passar pela controladora antes de gerar um novo;
 - **Tempo mínimo entre tokens** — intervalo mínimo entre passagens do token pela controladora; passagens abaixo desse intervalo indicam token duplicado.
-

Formato dos pacotes

DISCOVER

```
10:<apelido da origem>:<endereço IP da origem>
```

Exemplo: 10:A:10.32.143.20

HELLO

```
20:<apelido da origem>:<endereço IP da origem>:<CRC32>
```

Exemplo: 20:B:10.32.143.21:48291734

O CRC32 é calculado com o campo CRC vazio (terminado em :):

```
20:B:10.32.143.21:
```

HELLOs com CRC inválido são descartados silenciosamente. Os parâmetros do CRC32 são os mesmos definidos na seção de pacote de dados.

Token

```
1000
```

Um pacote com tipo 1000 que contenha campos adicionais (separados por :) **não deve ser tratado como token** e deve ser descartado imediatamente. Isso protege contra corrupção do tipo de pacote de dados (2000 → 1000): o descarte no primeiro salto dispara o timeout do token na controladora.

Pacote de dados

```
2000:<origem>:<destino>:<flag>:<número de sequência>:<TTL>:<mensagem>:<CRC32>
```

Exemplo: 2000:B:A:maquinainexistente:42:8:Oi pessoal!:19385749

Campos:

- **flag:** estado do pacote. Valores possíveis: `maquinainexistente`, `ACK`, `NAK`.
- **número de sequência:** inteiro incremental por máquina de origem, iniciando em 0 e incrementado a cada nova mensagem. Permanece fixo durante toda a circulação, inclusive em retransmissões por NAK.
- **TTL (*time to live*):** número máximo de saltos restantes. A origem define o TTL como o **dobro do número de máquinas no anel** (cobre ida e volta). Cada salto intermediário decrementa o TTL em 1. O destino, ao alterar a flag para ACK ou NAK, reseta o TTL para o dobro do número de máquinas que conhece. Pacote com TTL igual a 0 é descartado, disparando o timeout do token.
- **CRC32:** calculado com o campo CRC vazio (terminado em :). Cobre todos os campos anteriores: tipo, origem, destino, flag, número de sequência, TTL e mensagem.

Atenção: a mensagem e o apelido não podem conter dois pontos.

Verificação do CRC em todo salto: toda máquina que recebe um pacote de dados deve verificar o CRC antes de encaminhar. CRC inválido → descartar imediatamente (dispara timeout do token). Ao modificar qualquer campo (flag ou TTL), a máquina deve recomputar o CRC antes de reenviar.

Parâmetros do CRC32: deve-se utilizar o padrão **CRC32/ISO-HDLC** (também conhecido como CRC32b): polinômio `0xEDB88320` (forma refletida), seed inicial `0xFFFFFFFF`, XOR final `0xFFFFFFFF`, com reflexão de entrada e saída. Este é o comportamento padrão de `binascii.crc32` (Python), `java.util.zip.CRC32` (Java) e `crc32` via `zlib` (C). O resultado deve ser expresso como inteiro decimal sem sinal de 32 bits.

Inicialização do anel

Ao iniciar, a aplicação envia **DISCOVER** em broadcast se identificando. Toda máquina que receber um DISCOVER responde com **HELLO** em broadcast.

A máquina aguarda **1 segundo** após enviar o DISCOVER para coletar HELLOs. Ao fim desse período, constrói a lista de máquinas em ordem alfabética e determina seu sucessor. A **primeira máquina em ordem alfabética** gera o token inicial e passa a ser a **controladora do token** (responsável pelo timeout e pelo tempo mínimo).

Caso nenhum HELLO seja recebido dentro de 10 segundos, o DISCOVER é reenviado. Se a máquina estiver sozinha (nenhum HELLO recebido em 1 segundo), ela aguarda a entrada de outra máquina sem gerar token.

Heartbeat: cada máquina reenvia seu HELLO a cada **10 segundos**. Se uma máquina ficar **30 segundos** (3 ciclos) sem receber um HELLO válido de um host conhecido, esse host é considerado inativo e removido do anel. O anel é reconstruído automaticamente com os hosts restantes — nenhum protocolo explícito de saída é necessário.

Funcionamento do anel

Circulação do token

Ao receber o token, a máquina verifica sua fila de mensagens:

- **Fila vazia:** encaminha o token para o sucessor.
- **Fila com mensagens:** retira a primeira mensagem da fila, envia o pacote de dados para o sucessor e **aguarda o retorno** antes de encaminhar o token.

Envio de dados

O pacote de dados parte da origem com a flag `maquinainexistente`. Cada máquina intermediária verifica o CRC, decrementa o TTL, recomputa o CRC e encaminha para o sucessor. A máquina destino processa o pacote e o reenvia com flag `ACK` ou `NAK`. O pacote percorre o restante do anel até voltar à origem.

No destino

Ao identificar que o pacote é endereçado a ela:

1. Verificar o CRC. Se inválido, marcar a flag como **NAK**, recomputar o CRC e reenviar.

2. Se o CRC for válido, verificar o número de sequência. Cada máquina mantém o próximo número de sequência esperado para cada origem:
 - Se o número já foi recebido (duplicata): descartar o conteúdo, responder com **ACK**.
 - Se for o esperado: imprimir o apelido da origem e a mensagem, avançar o contador esperado, marcar flag como **ACK**.
3. Resetar o TTL para o dobro do número de máquinas conhecidas, recomputar o CRC e encaminhar para o sucessor.

Na origem

Ao receber o pacote de volta (campo **origem** igual ao próprio apelido), revalidar o CRC. CRC inválido é tratado como **NAK**.

- **ACK**: exibir mensagem na tela, retirar a mensagem da fila, encaminhar o token para o sucessor.
- **maquinainexistente**: a máquina destino não existe ou está inativa. Exibir mensagem na tela, retirar a mensagem da fila, encaminhar o token.
- **NAK**: a entrega falhou. Exibir mensagem na tela. Manter a mensagem na fila com o mesmo número de sequência e retransmitir na próxima passagem do token (encaminhar o token agora).

Módulo de inserção de falhas

Antes de enviar um pacote de dados, a máquina pode corromper artificialmente a mensagem com probabilidade configurável (definida no arquivo de configuração). A corrupção altera o conteúdo da mensagem, tornando o CRC inválido no receptor. O CRC é sempre inserido **após** a possível corrupção.

Fila de mensagens

Cada máquina mantém uma fila de até **10 mensagens**. O usuário enfileira mensagens via terminal (stdin) durante a execução, informando o apelido do destino e o conteúdo. Mensagens são enviadas em ordem FIFO e só são removidas da fila após confirmação (ACK ou **maquinainexistente**).

Controle do token

A controladora do token (a máquina com menor apelido alfabético no anel) monitora dois problemas:

- **Token perdido** — o token não passa pela controladora dentro do *timeout* configurado: um novo token é gerado.
- **Token duplicado** — o token passa em intervalo menor que o tempo mínimo configurado: o token extra é descartado.

Quando uma nova máquina com apelido anterior ao da controladora atual entra no anel, ela assume o controle: gera um novo token e passa a monitorar *timeout* e tempo mínimo. A controladora anterior cessa o monitoramento imediatamente ao perceber a troca. O mecanismo de tempo mínimo da nova controladora descarta o token duplicado automaticamente.

Alteração topológica do anel

O anel pode ser reconfigurado durante a execução sem reinicialização.

Entrada de nova máquina: a nova máquina envia DISCOVER em broadcast e aguarda 1 segundo pelos HELLOs. Todas as máquinas que recebem o DISCOVER respondem com HELLO e reconstróem o encadeamento do anel em ordem alfabética com base nos hosts conhecidos. A nova máquina só começa a encaminhar pacotes após esse período de espera.

Saída de máquina: detectada automaticamente pelo mecanismo de heartbeat (30 segundos sem HELLO válido). O anel é reconstruído com os hosts restantes sem nenhuma ação adicional.

Avaliação

A demonstração deverá acontecer, no mínimo, em 3 máquinas.

#	Critério	Descrição	Peso
1	Inicialização do anel	DISCOVER/HELLO via broadcast, formação da topologia em ordem alfabética, geração do token inicial	10%
2	Funcionamento básico do anel	Circulação do token, envio e retorno de pacotes de dados, fila de mensagens (até 10)	15%
3	CRC32	Cálculo e inserção do CRC na origem, verificação em todo salto, resposta com ACK/NAK/máquina inexistente	15%
4	Módulo de inserção de falhas	Inserção aleatória de erros com probabilidade configurável, retransmissão na próxima passagem do token	10%
5	Controle do token	Deteção de token perdido (timeout → gerar novo) e de token duplicado (tempo mínimo → retirar)	15%
6	Alteração topológica	Entrada e saída de máquinas durante execução sem reinicialização, reconfiguração via DISCOVER/HELLO/heartbeat	10%
7	Observabilidade	Visualização em tempo real da posição do token e pacotes; logs de ACK/NAK/retransmissão/token perdido/duplicado	5%
8	Relatório	Estruturas de dados, threads, sincronização, CRC, exemplos de execução, código comentado	20%
Total			100%

Atenção: O formato dos pacotes (DISCOVER, HELLO, token, dados) deve ser seguido fielmente, pois grupos diferentes se comunicarão durante a apresentação.

Regras Gerais

Grupos: Até 3 componentes. Informar no Moodle.

Data de entrega: 30/06 antes do início da aula

Data de apresentação: 30/06

Data de teste de interoperabilidade: 02/07

Obs.: Todos os participantes devem estar presentes

Entrega final:

- Relatório descrevendo a estrutura da solução dada, envolvendo estruturas de dados, threads, classes, mecanismos de sincronização utilizados, CRC, exemplos de execução, etc.
- Código fonte comentado.

IMPORTANTE: Não serão aceitos trabalhos entregues fora do prazo. Trabalhos que não compilam ou que não executam não serão avaliados. Todos os trabalhos serão analisados e comparados. Caso seja identificada cópia de trabalhos, todos os trabalhos envolvidos receberão nota ZERO.